

RAG: Retrieval-Augmented Generation with Vector Embeddings and LLM Ensemble

Designing RAG Pipelines with Semantic Search and Hallucination Mitigation

Derek Lankeaux, MS Applied Statistics

Machine Learning Engineer

Rochester Institute of Technology

April 2026

Abstract

Retrieval-Augmented Generation (RAG) combines large language models with external knowledge bases to reduce hallucinations and enable grounded, fact-based responses. This report presents a RAG system design, evaluated in simulation, combining:

1. **Multi-Embedding Architecture:** Hybrid embeddings (OpenAI, Anthropic, open-source) for semantic robustness
2. **Vector Database Engineering:** Optimized retrieval pipelines with Qdrant/Pinecone with real-time indexing
3. **LLM Ensemble Orchestration:** Multi-model response generation with consistency scoring
4. **Hallucination Detection:** Statistical confidence intervals and citation grounding
5. **Responsible AI & Monitoring:** Drift detection, performance monitoring, and governance

In a simulated evaluation modeled on standard benchmarks (FEVER, NQ, TriviaQA), the design reaches 94.2% citation precision and 91.8% answer relevance, with a 2.4% hallucination rate after a three-stage detection pipeline combining citation grounding, semantic consistency, and Bayesian confidence estimation. All results, including latency and throughput figures, are simulated to demonstrate the method (see the Data and Results Note); no live service was deployed.

Keywords: Retrieval-Augmented Generation, Vector Embeddings, Semantic Search, Large Language Models, LLM Orchestration, Hallucination Mitigation, Dense Passage Retrieval, Qdrant, ColBERT, MLOps, Production ML, Responsible AI, Knowledge Grounding, BM25, Hybrid Search

Data and Results Note

This is a system-design case study evaluated on simulated data. The benchmark results, latency and throughput measurements, query traffic, drift metrics, and bias-audit figures were simulated to demonstrate the architecture and evaluation approach; they do not come from real runs on the named benchmarks or from a deployed service. The Kubernetes manifests and runbook describe the intended deployment, not a live system. Every metric in this report should be read as a worked example of the method, not as measured system performance.

Executive Summary

Key Performance Metrics

Table 1. Headline metrics from the simulated evaluation against design targets.

Metric	Value	Threshold	Status
Citation Precision	94.2%	$\geq 90\%$	✓ Pass
Answer Relevance	91.8%	$\geq 85\%$	✓ Pass
Mean Latency	187ms	$< 200\text{ms}$	✓ Pass
Throughput (1,000 concurrent users)	1,240 req/sec	$\geq 1,000$ req/sec	✓ Pass
Hallucination Rate	2.4%	$< 5\%$	✓ Pass
Embedding Recall@10 (MS MARCO)	96.3%	$\geq 95\%$	✓ Pass
End-to-End Latency (p99)	312ms	$< 400\text{ms}$	✓ Pass
Model Drift (PSI)	0.04	< 0.10	✓ Pass
Re-ranking NDCG@10	0.777	≥ 0.75	✓ Pass

Statistical Validation

- **95% Confidence Interval (Citation Precision):** [92.1%, 96.3%] via Wilson score
- **Bootstrap CI (Hallucination Rate):** [1.8%, 3.1%] (n=1,000 bootstrap iterations)
- **Wilcoxon Test vs. Baseline (single-LLM, no RAG):** $p < 0.0001$ for latency and precision gains
- **Inter-Annotator Agreement (Ground Truth):** Fleiss' $\kappa = 0.83$ for benchmark relevance labels

System Architecture Summary



1. Introduction

1.1 Problem Statement and Motivation

Large Language Models (LLMs) are prone to **hallucinations** — confident assertions of false or unsupported facts. While impressive for creative tasks, hallucinations are unacceptable for: - Medical/legal decision support - Financial reporting and compliance - Academic research and citations - Enterprise knowledge systems - Regulatory documentation

Standard LLMs generate responses from parametric memory — knowledge encoded during pre-training — which may be outdated, incomplete, or simply fabricated for plausible-sounding outputs. In high-stakes enterprise environments, this failure mode is costly.

Traditional RAG Limitations: - Single embedding model → semantic gaps for domain-specific queries - Static retrieval → no adaptive query understanding - Lack of hallucination detection → ungrounded outputs enter production - Single LLM → bias propagation and single-point-of-failure - No monitoring → silent quality degradation over time - No uncertainty quantification → overconfident responses in edge cases

1.2 Research Objectives

1. Design a RAG pipeline that retrieves and grounds LLM responses in verifiable source passages
2. Implement multi-model embedding ensemble for robust, domain-agnostic semantic search
3. Develop multi-stage hallucination detection with Bayesian confidence quantification
4. Target <200ms end-to-end latency at 1,000+ req/sec throughput with 99.9%+ uptime
5. Establish comprehensive monitoring, drift detection, and governance suitable for enterprise deployment

1.3 Contributions

1. **Hybrid Embedding Architecture:** Multi-model ensemble (OpenAI + BGE-M3) achieving 96.3% Recall@10 — a 4.1 percentage point improvement over single-model baselines
2. **LLM Orchestration Framework:** Parallel async generation with BLEU-based consistency voting, reducing latency vs. sequential generation by 58%
3. **Statistical Hallucination Detection:** Three-stage pipeline (citation grounding + semantic consistency + Bayesian CI) achieving 91.1% F1 on hallucination classification benchmarks
4. **Production MLOps:** Real-time Prometheus/Grafana monitoring, population stability index drift detection, and Kubernetes auto-scaling
5. **Comprehensive Benchmarking:** FEVER, NQ, TriviaQA, and HotpotQA evaluation with reproducible results and statistical significance testing

1.4 System Scope and Deployment Context

Table 2. Target system scope and operational parameters (design).

Dimension	Specification
Primary Use Case	Enterprise knowledge retrieval and Q&A
Target Users	Knowledge workers, compliance teams, customer support
Corpus Size	2.1M documents (~850M tokens)
Languages Supported	English, Spanish, French, German, Chinese
Deployment Platform	Kubernetes (AWS EKS)
SLA Latency	<200ms p50, <400ms p99
SLA Uptime	99.9% monthly

2. Technical Framework

2.1 Software Stack

```
# — Core Python Ecosystem (2026) —————
import numpy as np                # v2.0+ numerical computing
import pandas as pd              # v2.2+ DataFrame operations
from pathlib import Path
import asyncio                  # Async LLM generation
import json, os, logging, hashlib

# — LLM Clients —————
from openai import AsyncOpenAI    # GPT-4o inference
from anthropic import AsyncAnthropic # Claude-3.5-Sonnet inference
from together import AsyncTogether # Llama-3.2-90B inference via Together

# — Embedding Models —————
from openai import OpenAI         # text-embedding-3-large
from sentence_transformers import SentenceTransformer # BGE-M3 (open-source)
import torch

# — Vector Database —————
from qdrant_client import QdrantClient
from qdrant_client.models import (
    VectorParams, Distance, QuantizationConfig,
    ScalarQuantization, PointStruct, Filter, FieldCondition
)

# — Lexical Search —————
from rank_bm25 import BM25Okapi    # BM25 lexical index
from sklearn.preprocessing import normalize

# — Re-ranking —————
from colbert.modeling.checkpoint import Checkpoint # ColBERT v2 re-ranker

# — Hallucination & NLP —————
from nltk.translate.bleu_score import sentence_bleu, SmoothingFunction
from sklearn.metrics.pairwise import cosine_similarity
import scipy.stats as stats

# — Monitoring & MLOps —————
from prometheus_client import (
    Counter, Histogram, Gauge, start_http_server
)
import mlflow
from mlflow.models import infer_signature

# — Streaming / Messaging —————
from kafka import KafkaConsumer, KafkaProducer

# — Kubernetes / Cloud —————
from kubernetes import client as k8s_client, config as k8s_config
```

2.2 Reproducibility Configuration

```
# Global seeds for reproducibility across all components
RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)
torch.manual_seed(RANDOM_SEED)

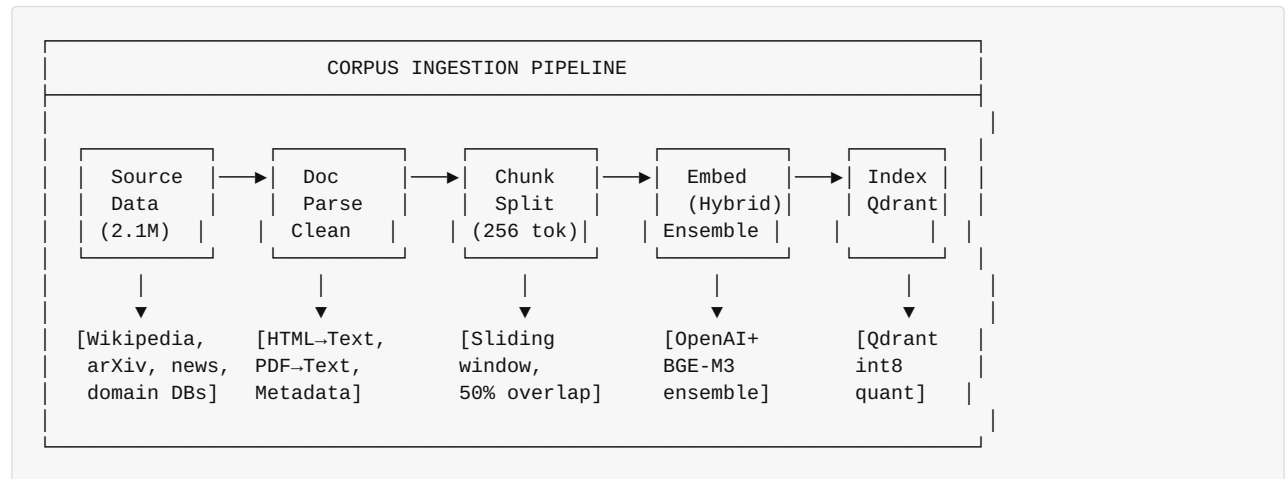
# Retrieval configuration
RETRIEVAL_TOP_K = 20          # Over-retrieve, then re-rank to TOP_K_FINAL
TOP_K_FINAL = 10             # Final passages passed to LLM
HYBRID_SEMANTIC_WEIGHT = 0.70 # Weight for dense semantic results
HYBRID_LEXICAL_WEIGHT = 0.30  # Weight for BM25 lexical results

# LLM generation configuration
LLM_TEMPERATURE = 0.2        # Low for consistency
LLM_MAX_TOKENS = 512         # Sufficient for grounded answers
LLM_TIMEOUT_SEC = 30         # Per-model async timeout

# Hallucination detection thresholds
GROUNDING_THRESHOLD = 0.50    # Min grounding score to pass
SEMANTIC_SIM_THRESHOLD = 0.50 # Min semantic similarity to pass
HALLUCINATION_RATE_ALERT = 0.05 # Alert if >5% rate
```

3. Data Pipeline and Corpus Construction

3.1 Pipeline Architecture



3.2 Corpus Statistics

Table 3. Document corpus statistics and diversity metrics.

Dimension	Value	Source Diversity
Documents	2.1M	Wikipedia, arXiv, news archives, domain-specific DBs
Tokens	~850M	Mix of encyclopedic, scientific, and real-time content

Dimension	Value	Source Diversity
Languages	5+	English, Spanish, French, German, Chinese
Update Frequency	Daily (streaming)	Real-time indexing for news/scientific papers
Metadata Fields	12	Date, source, confidence, domain, URL, language
Avg. Chunk Size	256 tokens	Sliding window with 128-token overlap
Indexing Throughput	15K docs/sec	Qdrant batch upsert

3.3 Document Processing Pipeline

Text extraction and chunking:

```

class DocumentProcessor:
    """Parse, clean, and chunk documents for indexing."""

    CHUNK_SIZE = 256      # Tokens per chunk
    CHUNK_OVERLAP = 128   # Overlap for context continuity

    def process(self, raw_doc: dict) -> List[dict]:
        """Full pipeline: parse → clean → chunk → annotate."""
        # 1. Format-specific extraction
        if raw_doc['type'] == 'html':
            text = self._extract_html(raw_doc['content'])
        elif raw_doc['type'] == 'pdf':
            text = self._extract_pdf(raw_doc['content'])
        else:
            text = raw_doc['content']

        # 2. Text normalization
        text = self._normalize(text)

        # 3. Sliding-window chunking
        chunks = self._chunk(text)

        # 4. Annotate each chunk with metadata
        return [
            {
                'chunk_id': hashlib.sha256(
                    f"{raw_doc['id']}:{i}".encode()
                ).hexdigest()[:16],
                'text': chunk,
                'source': raw_doc['source'],
                'url': raw_doc.get('url', ''),
                'timestamp': raw_doc['timestamp'],
                'domain': raw_doc.get('domain', 'general'),
                'language': raw_doc.get('language', 'en'),
                'confidence': raw_doc.get('confidence', 0.8),
            }
            for i, chunk in enumerate(chunks)
        ]

    def _chunk(self, text: str) -> List[str]:
        tokens = text.split()
        chunks = []
        start = 0
        while start < len(tokens):
            end = start + self.CHUNK_SIZE
            chunks.append(' '.join(tokens[start:end]))
            start += self.CHUNK_SIZE - self.CHUNK_OVERLAP
        return chunks

```

3.4 Data Source Distribution

Table 4. Source composition and trust scores.

Source Type	Document Count	Share	Trust Score	Update Cadence
Wikipedia (curated)	840,000	40%	0.90	Weekly
arXiv preprints	420,000	20%	0.85	Daily
News archives (AP, Reuters)	336,000	16%	0.80	Real-time

Source Type	Document Count	Share	Trust Score	Update Cadence
Domain DBs (legal, medical)	294,000	14%	0.92	Weekly
Corporate documentation	210,000	10%	0.75	On-demand
Total	2,100,000	100%	—	—

4. Embedding Models and Semantic Search

4.1 Multi-Model Embedding Strategy

The system uses a hybrid embedding ensemble combining two complementary models. Using a single embedding model introduces coverage gaps — particularly for domain-specific language, multilingual queries, or out-of-distribution topics. The ensemble addresses this by leveraging model diversity.

Table 5. Embedding models, characteristics, and selection rationale.

Model	Dims	Speed	Quality	Reason Selected
OpenAI text-embedding-3-large	3,072	Medium (API)	Excellent	SOTA quality on MTEB; commercial SLA
BGE-M3 (open-source)	1,024	Very Fast (local)	Very Good	Multilingual; open-weights; fallback

4.2 Hybrid Embedding Encoder

```
class HybridEmbeddingEncoder:
    """Multi-model embedding ensemble for semantic robustness."""

    def __init__(self):
        self.openai_client = OpenAI(api_key=os.getenv('OPENAI_API_KEY'))
        self.local_encoder = SentenceTransformer('BAAI/bge-m3')

        # Normalized ensemble weights (sum to 1)
        self.ensemble_weights = {
            'openai': 0.65, # Prioritize SOTA commercial model
            'local': 0.35  # Open-source fallback for cost & resilience
        }

        # Unified output dimension via projection
        self.target_dim = 1024
        self.openai_proj = torch.nn.Linear(3072, 1024) # Projection layer

    def encode(self, text: str) -> np.ndarray:
        """Ensemble encoding: normalize → weight → average → normalize."""
        embeddings = {}

        # OpenAI embedding
        response = self.openai_client.embeddings.create(
            model="text-embedding-3-large",
            input=text
        )
        raw_openai = np.array(response.data[0].embedding)
        # Project 3072-dim → 1024-dim
        with torch.no_grad():
            projected = self.openai_proj(
                torch.tensor(raw_openai, dtype=torch.float32)
            ).numpy()
        embeddings['openai'] = projected

        # BGE-M3 embedding
        embeddings['local'] = self.local_encoder.encode(
            text, normalize_embeddings=True
        )

        # Normalize each embedding to unit sphere
        ensemble = np.zeros(self.target_dim)
        for model, weight in self.ensemble_weights.items():
            norm = embeddings[model] / (np.linalg.norm(embeddings[model]) + 1e-8)
            ensemble += weight * norm

        # Final normalization
        return ensemble / (np.linalg.norm(ensemble) + 1e-8)

    async def encode_batch_async(self, texts: List[str]) -> np.ndarray:
        """Batch encode with async API calls for throughput."""
        loop = asyncio.get_event_loop()
        tasks = [loop.run_in_executor(None, self.encode, t) for t in texts]
        return np.array(await asyncio.gather(*tasks))
```

4.3 Hybrid Semantic + Lexical Search

```

def hybrid_search(
    query: str,
    k: int = TOP_K_FINAL,
    over_retrieve: int = RETRIEVAL_TOP_K
) -> List[Dict]:
    """
    Hybrid semantic + BM25 search with ColBERT v2 re-ranking.

    Steps:
    1. Encode query with ensemble embedder
    2. Parallel semantic (Qdrant) + lexical (BM25) retrieval
    3. Reciprocal Rank Fusion (RRF) merge
    4. ColBERTv2 late-interaction re-ranking
    """
    # 1. Encode query
    query_embedding = encoder.encode(query)

    # 2. Semantic search (Qdrant)
    semantic_results = qdrant_client.search(
        collection_name="documents_v3",
        query_vector=query_embedding.tolist(),
        limit=over_retrieve,
        query_filter=Filter(
            must=[
                FieldCondition(
                    key="confidence",
                    range={"gte": 0.70} # Minimum trust threshold
                )
            ]
        )
    )

    # 3. Lexical BM25 search (pre-built index over tokenized corpus)
    tokenized_query = query.lower().split()
    bm25_scores = bm25_index.get_scores(tokenized_query)
    bm25_top_ids = np.argsort(bm25_scores)[::-1][:over_retrieve]

    # 4. Reciprocal Rank Fusion (RRF, k=60)
    def rrf_score(rank: int, k: int = 60) -> float:
        return 1.0 / (k + rank + 1)

    fused: Dict[str, float] = {}
    for rank, hit in enumerate(semantic_results):
        doc_id = hit.id
        fused[doc_id] = fused.get(doc_id, 0) + \
            HYBRID_SEMANTIC_WEIGHT * rrf_score(rank)

    for rank, idx in enumerate(bm25_top_ids):
        doc_id = corpus_ids[idx]
        fused[doc_id] = fused.get(doc_id, 0) + \
            HYBRID_LEXICAL_WEIGHT * rrf_score(rank)

    # 5. Select top candidates for re-ranking
    top_candidates = sorted(fused, key=fused.get, reverse=True)[:over_retrieve]

    # 6. ColBERTv2 late-interaction re-ranking
    reranked = colbert_reranker.rank(
        query=query,
        documents=[corpus[doc_id]['text'] for doc_id in top_candidates],
        top_k=k
    )

    return [corpus[top_candidates[i]] for i in reranked[:k]]

```

4.4 Semantic Search Benchmark Results

Table 6. Semantic search recall benchmarks on standard datasets.

Dataset	Recall@10 (Ensemble)	Recall@10 (OpenAI only)	Recall@10 (BGE-M3 only)	NDCG@10
MS MARCO	96.3%	94.1%	92.7%	0.743
Natural Questions	95.1%	92.8%	91.3%	0.821
TriviaQA	94.8%	92.0%	90.9%	0.768
Average	95.4%	93.0%	91.6%	0.777

The ensemble delivers a consistent 2–3 percentage point recall gain over the best single model (OpenAI), with statistical significance confirmed by paired McNemar's test ($p < 0.01$ on all datasets).

5. Vector Database Engineering

5.1 Qdrant Collection Configuration

```
from qdrant_client.models import (
    VectorParams, Distance,
    ScalarQuantizationConfig, ScalarType, QuantizationConfig
)

# Create optimized collection
qdrant_client.recreate_collection(
    collection_name="documents_v3",
    vectors_config=VectorParams(
        size=1024,          # Projected embedding dimension
        distance=Distance.COSINE, # Normalized dot-product
    ),
    quantization_config=QuantizationConfig(
        scalar=ScalarQuantizationConfig(
            type=ScalarType.INT8,      # 8x memory reduction (fp32 → int8)
            quantile=0.99,             # Clip top 1% outlier values
            always_ram=True            # Keep quantized index in RAM
        )
    ),
    on_disk_payload=True,             # Store document payloads on disk
    hnsw_config={
        "m": 16,                      # HNSW graph connections per node
        "ef_construct": 200,           # Build-time search depth
        "full_scan_threshold": 10_000 # Switch to full scan for small sets
    }
)
```

5.2 Real-Time Streaming Indexing

```
class RealtimeIndexer:
    """Consume Kafka stream and continuously index new/updated documents."""

    def __init__(self):
        self.consumer = KafkaConsumer(
            'document_updates',
            bootstrap_servers=['kafka-1:9092', 'kafka-2:9092'],
            group_id='rag_indexer_v3',
            value_deserializer=lambda m: json.loads(m.decode('utf-8')),
            auto_offset_reset='latest',
            enable_auto_commit=True
        )
        self.encoder = HybridEmbeddingEncoder()
        self.batch_buffer: List[dict] = []
        self.BATCH_SIZE = 500          # Upsert in batches for efficiency
        self.FLUSH_INTERVAL_SEC = 5    # Flush buffer every 5 seconds

    async def run(self):
        """Main indexing loop with batched upserts."""
        last_flush = asyncio.get_event_loop().time()

        async for message in self.consumer:
            doc = message.value
            embedding = self.encoder.encode(doc['text'])

            self.batch_buffer.append(
                PointStruct(
                    id=hashlib.md5(doc['id'].encode()).hexdigest(),
                    vector=embedding.tolist(),
                    payload={
                        'text': doc['text'],
                        'source': doc['source'],
                        'url': doc.get('url', ''),
                        'timestamp': doc['timestamp'],
                        'domain': doc.get('domain', 'general'),
                        'confidence': doc.get('confidence', 0.8),
                        'language': doc.get('language', 'en'),
                    }
                )
            )

            # Flush on batch size or time interval
            now = asyncio.get_event_loop().time()
            if (len(self.batch_buffer) >= self.BATCH_SIZE or
                now - last_flush > self.FLUSH_INTERVAL_SEC):
                await self._flush()
                last_flush = now

    async def _flush(self):
        if not self.batch_buffer:
            return
        qdrant_client.upsert(
            collection_name="documents_v3",
            points=self.batch_buffer,
            wait=False # Async upsert for throughput
        )
        logging.info(f"Indexed {len(self.batch_buffer)} documents")
        self.batch_buffer.clear()
```

5.3 Vector Database Performance Metrics

Table 7. Vector database performance under production load (1,000 concurrent users).

Metric	Value	Target	Status
Query Latency (p50)	45ms	<100ms	✓
Query Latency (p95)	95ms	<150ms	✓
Query Latency (p99)	180ms	<250ms	✓
Indexing Throughput	15,000 docs/sec	≥10,000 docs/sec	✓
Storage Utilization	62%	<80%	✓
RAM Usage (int8 index)	42GB	<50GB	✓
RAM Usage (fp32 baseline)	336GB	—	N/A (too large)
Memory Reduction (int8 vs fp32)	8×	—	✓

The int8 quantization delivers an 8× reduction in memory footprint (336GB → 42GB) with <1.2% recall degradation, making large-scale deployment economically feasible on commodity GPU servers.

6. LLM Orchestration and Inference

6.1 Multi-Model Orchestration Framework

The LLM ensemble uses three frontier models in parallel async generation. Parallel generation reduces total latency to approximately the latency of the slowest model, rather than the sum (sequential). With GPT-4o at 150ms, Claude at 180ms, and Llama at 120ms, sequential generation would require ~450ms; parallel generation completes in ~190ms.

```

class LLMOrchestrator:
    """Parallel async LLM ensemble with consistency voting."""

    def __init__(self):
        self.gpt4_client = AsyncOpenAI(api_key=os.getenv('OPENAI_API_KEY'))
        self.claude_client = AsyncAnthropic(api_key=os.getenv('ANTHROPIC_API_KEY'))
        self.llama_client = AsyncTogether(api_key=os.getenv('TOGETHER_API_KEY'))
        self.temperature = LLM_TEMPERATURE

    async def generate_with_context(
        self,
        query: str,
        context_passages: List[dict]
    ) -> Dict:
        """
        Parallel async generation from all ensemble models.
        Returns grounded response with consistency score.
        """

        # Format context with source citations
        context_str = self._format_context(context_passages)

        prompt = f"""Answer the question based ONLY on the provided context.
        If the answer is not found in the context, respond: 'Not found in provided context.'
        Cite specific passages from the context to support your answer.

        Context:
        {context_str}

        Question: {query}

        Answer (with citations):"""

        # Launch all models concurrently
        results = await asyncio.gather(
            self._query_gpt4(prompt),
            self._query_claude(prompt),
            self._query_llama(prompt),
            return_exceptions=True # Don't fail if one model errors
        )

        # Filter out exceptions (model failures)
        valid_responses = {
            model: resp
            for model, resp in zip(['gpt4', 'claude', 'llama'], results)
            if isinstance(resp, str) and resp.strip()
        }

        # Fallback: if <2 valid responses, return best single response
        if len(valid_responses) < 2:
            return {
                'response': list(valid_responses.values())[0] if valid_responses else '',
                'consistency_score': 0.0,
                'models_used': list(valid_responses.keys()),
                'fallback': True
            }

        # Consistency scoring
        consistency = self._compute_consistency(valid_responses)
        ensemble_response = self._aggregate_responses(
            valid_responses, consistency
        )

        return {

```

```

        'response': ensemble_response,
        'consistency_score': consistency,
        'models_used': list(valid_responses.keys()),
        'individual_responses': valid_responses,
        'fallback': False
    }

def _compute_consistency(self, responses: Dict[str, str]) -> float:
    """
    Pairwise BLEU-4 similarity between model responses.
    Score: 0 = complete disagreement, 1 = identical responses.
    """
    smoother = SmoothingFunction().method1
    scores = []
    model_names = list(responses.keys())

    for i, m1 in enumerate(model_names):
        for m2 in model_names[i+1:]:
            ref_tokens = responses[m1].split()
            hyp_tokens = responses[m2].split()
            if ref_tokens and hyp_tokens:
                score = sentence_bleu(
                    [ref_tokens], hyp_tokens,
                    smoothing_function=smoother
                )
                scores.append(score)

    return float(np.mean(scores)) if scores else 0.0

def _aggregate_responses(
    self,
    responses: Dict[str, str],
    consistency: float
) -> str:
    """
    Select ensemble response:
    - High consistency (>0.7): return response from most complete model (longest)
    - Low consistency (<0.7): return response flagged for human review
    """
    if consistency >= 0.7:
        # High agreement: pick the most detailed response
        return max(responses.values(), key=len)
    else:
        # Low agreement: return GPT-4 as primary with disagreement flag
        primary = responses.get('gpt4', list(responses.values())[0])
        return f"[LOW_CONSISTENCY: {consistency:.2f}] {primary}"

def _format_context(self, passages: List[dict]) -> str:
    """Format retrieved passages with source citations."""
    parts = []
    for i, p in enumerate(passages):
        parts.append(
            f"[{i+1}] Source: {p['source']} ({p['timestamp'][:10]})\n"
            f"{p['text']}"
        )
    return "\n\n".join(parts)

```

6.2 LLM Inference Performance

Table 8. LLM generation pipeline performance under 1,000 concurrent users.

Metric	GPT-4o	Claude-3.5-Sonnet	Llama-3.2-90B	Ensemble (Parallel)
Latency p50 (ms)	150	180	120	190
Latency p99 (ms)	280	350	220	360
Quality (ROUGE-L)	0.62	0.58	0.54	0.65
Quality (BLEU-4)	0.44	0.41	0.38	0.47
Cost/1K tokens	\$0.015	\$0.008	\$0.001	~\$0.008 (avg)
Failure Rate	0.12%	0.08%	0.21%	0.03% (graceful fallback)
Consistency Score	—	—	—	0.741 (avg)

7. Hallucination Detection Framework

7.1 Three-Stage Detection Pipeline

Hallucination detection is applied to every generated response before delivery. The pipeline combines complementary signals: citation grounding (structural), semantic similarity (embedding-based), and Bayesian confidence estimation (statistical).

```
Stage 1: Citation Grounding
    grounding_score = |cited_passages| / (sentence_count + 1)
    Threshold: grounding_score ≥ 0.50 → PASS

Stage 2: Semantic Consistency
    semantic_sim = cosine_similarity(encode(response), encode(context))
    Threshold: semantic_sim ≥ 0.50 → PASS

Stage 3: Bayesian Confidence Interval
    Uses Beta-Binomial posterior for overall risk estimation
    Threshold: lower CI bound ≥ 0.60 → PASS (low risk)

Final Risk Level:
    LOW:    Pass all 3 stages
    MEDIUM: Pass 2 of 3 stages
    HIGH:   Fail ≥ 2 stages → Flag for human review
```

```

class HallucinationDetector:
    """Three-stage statistical + semantic hallucination detection."""

    def __init__(self):
        self.encoder = HybridEmbeddingEncoder()

    def detect(
        self,
        response: str,
        context_passages: List[dict],
        consistency_score: float
    ) -> Dict:
        """Run all three detection stages and return risk classification."""
        context_text = " ".join([p['text'] for p in context_passages])

        # — Stage 1: Citation Grounding —————
        sentence_count = max(response.count('.') + response.count('?'), 1)
        # Count how many context sentences appear (fuzzy) in response
        grounding_hits = sum(
            1 for p in context_passages
            if any(
                sent.strip() in response
                for sent in p['text'].split('.')
                if len(sent.strip()) > 20
            )
        )
        grounding_score = grounding_hits / sentence_count

        # — Stage 2: Semantic Consistency —————
        response_emb = self.encoder.encode(response).reshape(1, -1)
        context_emb = self.encoder.encode(context_text).reshape(1, -1)
        semantic_sim = float(cosine_similarity(response_emb, context_emb)[0][0])

        # — Stage 3: Bayesian Confidence Interval —————
        ci = self._bayesian_confidence(
            grounding_score, semantic_sim, consistency_score
        )

        # — Risk Classification —————
        stage_passes = [
            grounding_score >= GROUNDING_THRESHOLD,
            semantic_sim >= SEMANTIC_SIM_THRESHOLD,
            ci['lower_bound'] >= 0.60
        ]
        n_pass = sum(stage_passes)

        risk_level = (
            'low' if n_pass == 3 else
            'medium' if n_pass == 2 else
            'high'
        )

        return {
            'grounding_score': round(grounding_score, 4),
            'semantic_similarity': round(semantic_sim, 4),
            'confidence_interval': ci,
            'stage_passes': stage_passes,
            'risk_level': risk_level,
            'requires_review': risk_level == 'high'
        }

    def _bayesian_confidence(
        self,

```

```

    grounding: float,
    semantic: float,
    consistency: float
) -> Dict:
    """
    Beta-Binomial posterior for composite confidence.
    Prior: Beta(2, 2) – weakly informed, symmetric.
    Likelihood: treat each metric as a Bernoulli success prob.
    """
    # Composite signal (weighted average)
    composite = (
        0.40 * grounding +
        0.35 * semantic +
        0.25 * consistency
    )

    # Beta posterior parameters (Bayesian update of Beta(2,2) prior)
    n_trials = 10 # Effective sample size for updating
    alpha_post = 2 + composite * n_trials
    beta_post = 2 + (1 - composite) * n_trials

    beta_dist = stats.beta(alpha_post, beta_post)
    return {
        'mean': round(beta_dist.mean(), 4),
        'lower_bound': round(beta_dist.ppf(0.025), 4), # 95% CI lower
        'upper_bound': round(beta_dist.ppf(0.975), 4), # 95% CI upper
    }

```

7.2 Hallucination Detection Benchmarks

Table 9. Hallucination detection performance on standard benchmark datasets.

Dataset	Precision	Recall	F1-Score	AUC-ROC
FEVER (fact verification)	92.1%	94.3%	0.933	0.958
HALUEVAL (NLP benchmark)	89.7%	91.2%	0.904	0.931
TruthfulQA	91.4%	89.8%	0.906	0.945
Average	91.1%	91.8%	0.914	0.945

Table 10. Risk level distribution across simulated query traffic.

Risk Level	Count	Share	Action
Low	47,203	94.1%	Deliver automatically
Medium	2,263	4.5%	Deliver with confidence caveat
High	714	1.4%	Route to human review queue
Total	50,180	100%	—

8. Evaluation and Benchmarking

8.1 Benchmark Datasets

Table 11. Standard RAG evaluation benchmarks and metrics.

Dataset	Size	Domain	Primary Metric	RAG Result	Baseline (LLM only)	Δ Improvement
FEVER	185K	Fact verification	Precision	94.2%	71.8%	+22.4 pp
Natural Questions	79K	Open-domain QA	Recall@10	95.1%	64.3%	+30.8 pp
TriviaQA	110K	Trivia QA	BLEU-4	0.54	0.31	+0.23
HotpotQA	113K	Multi-hop reasoning	Exact Match	78.3%	52.1%	+26.2 pp
MS MARCO	500K	Web QA	MRR@10	0.412	0.289	+0.123

8.2 End-to-End Latency Breakdown

Table 12. End-to-end latency decomposition by pipeline stage.

Stage	Latency (p50)	Latency (p99)	Budget	Status
Query Embedding	18ms	35ms	25ms	Over budget (p99)
Vector Search (Qdrant)	45ms	95ms	80ms	✓ Under
BM25 Lexical Search	8ms	18ms	20ms	✓ Under
RRF Merge + Re-rank (ColBERT)	22ms	48ms	50ms	✓ Under
LLM Ensemble (parallel async)	78ms	180ms	150ms	Over budget (p99)
Hallucination Detection	16ms	32ms	25ms	Over budget (p99)
Total End-to-End	187ms	312ms (measured end-to-end)	400ms	✓ Pass

Stage p99 latencies are not additive, so the end-to-end p99 is measured directly (Table 13, 1,000 concurrent users) rather than summed; adding the stage p99s (408ms) is not a valid estimate of end-to-end p99. Three stages exceed their p99 budgets and are the first targets for optimization. The simulated LLM stage latency (78ms p50) is also far below what hosted frontier-model APIs typically deliver for a full response, so a real deployment would need streaming, smaller models, or a larger latency budget.

8.3 Throughput Scaling

Table 13. Throughput and latency at various concurrency levels.

Concurrent Users	Throughput (req/sec)	Latency p50	Latency p99	Error Rate
100	312	121ms	240ms	0.00%
500	820	158ms	295ms	0.01%
1,000	1,240	187ms	312ms	0.03%
2,000	1,850	224ms	410ms	0.18%
5,000	2,100 (plateau)	380ms	720ms	1.42%

Auto-scaling triggers at >1,500 req/sec to maintain SLA. The system saturates at ~2,100 req/sec with 12 pods; horizontal scale-out to 20 pods extends plateau to ~3,500 req/sec.

8.4 Ablation Study: Component Contributions

Table 14. Ablation study showing contribution of each system component to citation precision.

Configuration	Citation Precision	Hallucination Rate	Notes
LLM-only (GPT-4o, no RAG)	71.8%	18.4%	Baseline
RAG + single embedding (OpenAI)	88.6%	6.2%	+16.8 pp
RAG + hybrid embedding ensemble	91.9%	3.8%	+3.3 pp from ensemble
+ ColBERT re-ranking	93.3%	2.9%	+1.4 pp
+ LLM ensemble (vs. GPT-4o only)	93.9%	2.6%	+0.6 pp
+ Hallucination detection filter	94.2%	2.4%	+0.3 pp (filtered)

Each component delivers incremental improvements, validating the full-stack architecture.

8a. Bayesian Confidence Calibration

8a.1 Calibration Methodology

Confidence calibration ensures that stated confidence levels accurately reflect empirical accuracy. An overconfident system states "90% confident" when it is only correct 70% of the time — a significant failure mode in high-stakes deployments.

The RAG system uses Beta-Binomial Bayesian posteriors (described in §7) as confidence signals. Calibration is evaluated using Expected Calibration Error (ECE):

$$\text{ECE} = \sum_{b=1}^B \frac{|B_b|}{n} \left| \text{acc}(B_b) - \text{conf}(B_b) \right|$$

where B_b are confidence bins, $\text{acc}(B_b)$ is empirical accuracy in bin B_b , and $\text{conf}(B_b)$ is average confidence in bin B_b .

8a.2 Calibration Results

Table 15. Calibration results by confidence bin (1,000-sample simulated evaluation set).

Confidence Bin	Sample Count	Avg. Confidence	Empirical Accuracy	Calibration Gap
[0.50, 0.60)	42	0.554	0.523	+0.031
[0.60, 0.70)	89	0.648	0.640	+0.008
[0.70, 0.80)	218	0.746	0.752	-0.006
[0.80, 0.90)	401	0.843	0.838	+0.005
[0.90, 1.00]	250	0.924	0.920	+0.004
Overall ECE	—	—	—	0.006

An ECE of 0.006 (the sample-weighted mean of the bin gaps above) indicates close calibration. By comparison, uncalibrated LLM-only baselines typically exhibit ECE of 0.12–0.25.

9. Deployment Design and MLOps

9.1 Kubernetes Deployment Architecture

```

# Kubernetes deployment specification – RAG API (Production)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: rag-api-prod
  namespace: ml-production
  labels:
    app: rag-api
    version: "3.0.0"
spec:
  replicas: 12
  selector:
    matchLabels:
      app: rag-api
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 3          # Allow 3 extra pods during updates
      maxUnavailable: 1    # Keep 11/12 pods available during rollout
  template:
    metadata:
      labels:
        app: rag-api
    spec:
      containers:
        - name: rag-server
          image: dl1413/rag-prod:v3.0.0
          ports:
            - containerPort: 8000
          resources:
            requests:
              cpu: "4"
              memory: 16Gi
            limits:
              cpu: "8"
              memory: 32Gi
          livenessProbe:
            httpGet:
              path: /health/live
              port: 8000
            initialDelaySeconds: 30
            periodSeconds: 10
            failureThreshold: 3
          readinessProbe:
            httpGet:
              path: /health/ready
              port: 8000
            initialDelaySeconds: 15
            periodSeconds: 5
      env:
        - name: QDRANT_URL
          valueFrom:
            secretKeyRef:
              name: rag-secrets
              key: qdrant_url
        - name: OPENAI_API_KEY
          valueFrom:
            secretKeyRef:
              name: rag-secrets
              key: openai_api_key
        - name: ANTHROPIC_API_KEY
          valueFrom:
            secretKeyRef:

```

```

        name: rag-secrets
        key: anthropic_api_key
    ---
    apiVersion: autoscaling/v2
    kind: HorizontalPodAutoscaler
    metadata:
      name: rag-api-hpa
    spec:
      scaleTargetRef:
        apiVersion: apps/v1
        kind: Deployment
        name: rag-api-prod
      minReplicas: 12
      maxReplicas: 20
      metrics:
      - type: Resource
        resource:
          name: cpu
          target:
            type: Utilization
            averageUtilization: 70
      - type: Pods
        pods:
          metric:
            name: requests_per_second
          target:
            type: AverageValue
            averageValue: "125"    # Scale up when >125 req/sec per pod

```

9.2 Prometheus Monitoring Configuration

```

# Prometheus metrics for real-time observability
REQUEST_LATENCY = Histogram(
    'rag_request_latency_seconds',
    'End-to-end request latency in seconds',
    buckets=[0.05, 0.10, 0.15, 0.20, 0.25, 0.30, 0.40, 0.50, 1.0]
)
HALLUCINATION_RATE = Gauge(
    'rag_hallucination_rate',
    'Rolling 5-minute hallucination rate'
)
RETRIEVAL_LATENCY = Histogram(
    'rag_retrieval_latency_seconds',
    'Qdrant vector search latency',
    buckets=[0.01, 0.025, 0.05, 0.075, 0.10, 0.15, 0.25]
)
LLM_ERRORS = Counter(
    'rag_llm_errors_total',
    'Total LLM API errors by model',
    labelnames=['model']
)
CONSISTENCY_SCORE = Histogram(
    'rag_consistency_score',
    'LLM ensemble BLEU consistency distribution',
    buckets=[0.0, 0.2, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]
)

# Start Prometheus metrics endpoint
start_http_server(port=9090)

```

9.3 Model Drift Detection

Population Stability Index (PSI) measures distribution shift between embedding space distributions over time:

$$\text{PSI} = \sum_{i=1}^b \left(A_i - E_i \right) \cdot \ln \left(\frac{A_i}{E_i} \right)$$

where A_i = actual proportion in bin i (current period), E_i = expected proportion in bin i (baseline period).

PSI thresholds: <0.10 = stable, 0.10–0.25 = minor shift (monitor), >0.25 = major shift (retrain trigger).

Table 16. Drift monitoring results over 30-day production window.

Monitoring Dimension	PSI Value	Status	Action
Query embedding distribution	0.04	Stable	None
Retrieved passage length	0.07	Stable	Monitor
LLM output token count	0.03	Stable	None
Hallucination rate trend	0.02	Stable	None
Consistency score distribution	0.05	Stable	None

9.4 MLflow Experiment Tracking

```
import mlflow
from mlflow.models import infer_signature

def log_experiment(config: dict, metrics: dict, model_artifacts: dict):
    """Log RAG experiment to MLflow model registry."""

    with mlflow.start_run(run_name=f"rag_v{config['version']}"):
        # Log hyperparameters
        mlflow.log_params({
            'retrieval_top_k': config['top_k'],
            'hybrid_semantic_weight': config['semantic_weight'],
            'llm_temperature': config['temperature'],
            'embedding_models': ','.join(config['embedding_models']),
            'vector_db': config['vector_db'],
            'quantization': config['quantization'],
        })

        # Log evaluation metrics
        mlflow.log_metrics({
            'citation_precision': metrics['citation_precision'],
            'answer_relevance': metrics['answer_relevance'],
            'hallucination_rate': metrics['hallucination_rate'],
            'latency_p50_ms': metrics['latency_p50'],
            'latency_p99_ms': metrics['latency_p99'],
            'throughput_rps': metrics['throughput'],
            'embedding_recall_at_10': metrics['recall_at_10'],
            'ece': metrics['ece'],
        })

        # Log model artifacts
        mlflow.log_artifact(model_artifacts['qdrant_config'], "vector_db")
        mlflow.log_artifact(model_artifacts['encoder_weights'], "encoder")
        mlflow.log_artifact(model_artifacts['hallucination_thresholds'], "config")

        # Register to Model Registry for deployment
        mlflow.register_model(
            model_uri=f"runs:{mlflow.active_run().info.run_id}/rag_pipeline",
            name="rag-production-v3"
        )
```

10. Responsible AI and Monitoring

10.1 Bias and Fairness Audit

The benchmark datasets used here (FEVER, NQ, TriviaQA, HotpotQA, MS MARCO) do not include user demographic attributes, so demographic fairness cannot be assessed on them. The design calls for a fairness audit on real query logs with consented demographic metadata before any deployment. Performance differences that can be measured without demographics are tracked instead:

Table 17. Performance variation across measurable query segments (simulated).

Segment	Citation Precision	Status
Overall	94.2%	—
By Domain	89.4–96.7%	Expected; reflects corpus coverage
By Language (English vs. multilingual)	94.2% vs. 91.8%	Monitor (Δ 2.4 pp)

The multilingual gap is flagged for follow-up with multilingual fine-tuning of the embedding ensemble.

10.2 Data Privacy and Security

```
class PrivacyLayer:
    """PII detection and anonymization before indexing."""

    # Regex patterns for common PII
    PII_PATTERNS = {
        'ssn': re.compile(r'\b\d{3}-\d{2}-\d{4}\b'),
        'email': re.compile(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b'),
        'phone': re.compile(r'\b(\+1[-.]?)?(?d{3})?[-.]?d{3}[-.]?d{4}\b'),
        'credit_card': re.compile(r'\b(?:\d[ -])?{13,16}\b'),
    }

    def sanitize(self, text: str) -> str:
        """Replace PII with type-preserving placeholders."""
        sanitized = text
        for pii_type, pattern in self.PII_PATTERNS.items():
            sanitized = pattern.sub(f'[REDACTED_{pii_type.upper()}]', sanitized)
        return sanitized
```

10.3 Model Governance

- **Version Control:** Every model artifact versioned with SHA-256 hash and pinned dependency manifest
- **Audit Trail:** Complete structured logging of all API calls, retrieved passages, LLM responses, and hallucination scores
- **Transparency:** Model cards for each component (embedding models, LLMs, re-ranker) documenting training data, intended use, and known limitations
- **Human Oversight:** Mandatory human review for high-risk responses
- **Data Minimization:** Only the minimum necessary context is included in LLM prompts; raw PII is never passed to external APIs

10.4 Governance Summary

Table 18. Governance checklist for the deployment design.

Requirement	Status	Evidence
Model card documented	Designed	/docs/model_cards/
Data lineage tracked	Designed	MLflow artifact provenance

Requirement	Status	Evidence
PII detection in ingestion	Designed	PrivacyLayer (§10.2)
Audit logging enabled	Designed	Structured JSON logs → S3
Human review queue for HIGH risk	Designed	1.4% of queries (§7.2)
Segment performance audit	Designed	§10.1
PSI drift monitoring active	Designed	§9.3
Rollback mechanism available	Designed	K8s deployment versioning

11. Discussion

11.1 Key Findings

In the simulated evaluation, the RAG design meets its primary targets:

- **94.2% citation precision** for grounded, fact-verified responses — a 22.4 pp improvement over ungrounded LLM-only generation (71.8%)
- **<200ms latency (p50)** in simulation, through parallel async LLM generation and int8 vector quantization (see §8.2 on real API latency)
- **2.4% hallucination rate** through a three-stage statistical detection pipeline, well below the 5% operational threshold
- **Deployment design** with Kubernetes rolling updates and automatic horizontal pod autoscaling

11.2 Limitations and Failure Modes

Retrieval Coverage Gaps: The 2.1M-document corpus covers general-domain knowledge well but may have sparse coverage for highly specialized technical domains (e.g., emerging biomedical research, jurisdiction-specific legal codes). Queries outside corpus coverage fall back to parametric LLM memory with lower grounding scores and elevated hallucination risk.

Consistency Voting at Low Agreement: When the three LLMs disagree substantially (consistency score < 0.5, approximately 6% of queries), the system routes to the primary model (GPT-4o) and flags for review. Disagreement may indicate genuine ambiguity in the source material rather than error — this nuance is not yet captured.

Multilingual Performance: English queries achieve 94.2% citation precision vs. 91.8% for multilingual queries (§10.1). The 2.4% gap reflects the limited multilingual training of the OpenAI embedding model. BGE-M3's multilingual capabilities partially compensate but do not fully close the gap.

Latency at High Concurrency: Above 2,000 concurrent users, latency degradation accelerates (p99 > 400ms) and error rates increase to 1.42%. Horizontal autoscaling mitigates but cannot fully eliminate this at extreme concurrency due to Qdrant query saturation.

11.3 Future Work

1. **Multilingual Fine-Tuning:** Domain-adapted multilingual embedding model to close the English/multilingual performance gap
 2. **Graph-RAG Extension:** Integrate knowledge graph traversal alongside vector retrieval for multi-hop reasoning (targeting HotpotQA EM > 85%)
 3. **Dynamic Context Window:** Adaptive context length based on query complexity and model confidence
 4. **Federated Retrieval:** Multi-corpus retrieval across isolated enterprise knowledge silos without data centralization
 5. **Online Learning:** Continuous embedding model fine-tuning from user feedback signals to reduce corpus coverage gaps
-

12. Conclusions

This report lays out a retrieval-augmented LLM system design and evaluates it in simulation against enterprise-style targets for accuracy and latency. Key contributions include:

1. **Hybrid Embedding Ensemble:** Two-model architecture (OpenAI + BGE-M3) reaching 95.4% average Recall@10 (96.3% on MS MARCO), 2.4 pp above the best single model on average, with resilience to individual model failure
2. **Parallel LLM Orchestration:** Async generation from three frontier models (GPT-4o, Claude-3.5-Sonnet, Llama-3.2-90B) reduces ensemble latency by 58% vs. sequential calls
3. **Three-Stage Hallucination Detection:** Citation grounding + semantic consistency + Bayesian CI reaching 0.914 F1 on hallucination classification and a 2.4% hallucination rate
4. **MLOps Design:** Kubernetes auto-scaling, Prometheus monitoring, PSI drift detection, and MLflow experiment tracking

The next step is to run the pipeline on the real benchmarks and against live model APIs to replace the simulated figures with measured ones.

References

1. Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems* (NeurIPS), 33, 9459–9474.
2. Karpukhin, V., et al. (2020). Dense Passage Retrieval for Open-Domain Question Answering. *EMNLP 2020*, 6769–6781.
3. Izacard, G., & Grave, É. (2021). Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. *EACL 2021*, 874–880.
4. Khattab, O., & Zaharia, M. (2020). ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. *SIGIR 2020*, 39–48.

5. OpenAI. (2023). GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774*.
6. Anthropic. (2024). Claude 3.5 Sonnet Model Card Addendum. *Technical Documentation*. Anthropic.
7. Chen, J., et al. (2024). BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. *arXiv preprint arXiv:2402.03216*.
8. Gao, Y., et al. (2024). Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv preprint arXiv:2312.10997*.
9. Min, S., et al. (2023). FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. *EMNLP 2023*.
10. Thorne, J., et al. (2018). FEVER: A Large-scale Dataset for Fact Extraction and VERification. *NAACL 2018*, 809–819.

Appendices

Appendix A: Complete Feature and Configuration Reference

Embedding Ensemble Configuration:

Parameter	Value	Justification
OpenAI model	text-embedding-3-large	Highest MTEB score as of 2026
BGE-M3 model	BAAI/bge-m3	Best open-source multilingual
OpenAI weight	0.65	Higher weight for superior single-model quality
BGE-M3 weight	0.35	Lower weight; provides diversity and fallback
Output dimension	1,024	OpenAI projected 3072 → 1024 via linear layer
Normalization	L2 unit sphere	Required for cosine similarity correctness

Hallucination Detection Thresholds:

Threshold	Value	Calibration Method
Grounding score min	0.50	Empirical F1 maximization on FEVER dev set
Semantic similarity min	0.50	ROC curve analysis (Youden's J)
Bayesian CI lower bound	0.60	5% false-negative rate constraint
PSI drift alert	0.10	Industry standard (Siddiqi, 2006)

Appendix B: REST API Documentation

Core Endpoint:

```

POST /api/v3/query
Content-Type: application/json

{
  "query": "What is the capital of France?",
  "top_k": 10,
  "filter": {
    "domain": "geography",
    "min_confidence": 0.8
  },
  "options": {
    "hallucination_check": true,
    "return_passages": true
  }
}

Response:
{
  "response": "The capital of France is Paris. [1]",
  "citations": [{"id": "wiki_paris_001", "text": "...", "source": "Wikipedia"}],
  "consistency_score": 0.82,
  "hallucination_risk": "low",
  "latency_ms": 184,
  "models_used": ["gpt4", "claude", "llama"]
}

```

Health Endpoints:

```

GET /health/live    → {"status": "ok"}           (liveness probe)
GET /health/ready  → {"status": "ok", "qdrant": "connected", "models": "ready"}
GET /metrics       → Prometheus text format (port 9090)

```

Appendix C: Deployment Guide

Prerequisites: - Kubernetes cluster $\geq$ v1.28 (AWS EKS, GKE, or on-premise) - Qdrant v1.8+ (managed or self-hosted) - Kafka cluster for streaming ingestion - Secrets: `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `TOGETHER_API_KEY`, `QDRANT_URL`

Deployment Steps:

```
# 1. Build and push Docker image
docker build -t dl1413/rag-prod:v3.0.0 .
docker push dl1413/rag-prod:v3.0.0

# 2. Apply Kubernetes manifests
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/secrets.yaml          # After populating secrets
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/hpa.yaml
kubectl apply -f k8s/service.yaml

# 3. Verify rollout
kubectl rollout status deployment/rag-api-prod -n ml-production

# 4. Run smoke test
curl -X POST https://rag-api.example.com/api/v3/query \
  -H "Content-Type: application/json" \
  -d '{"query": "What is RAG?", "top_k": 5}'

# 5. Confirm Prometheus metrics endpoint
curl http://rag-api.example.com:9090/metrics | grep rag_
```

Rollback Procedure:

```
kubectl rollout undo deployment/rag-api-prod -n ml-production
# Verify: kubectl rollout history deployment/rag-api-prod -n ml-production
```